

ACCOUNTING SYSTEM

OJT Static Demo Development Project Brief

Laravel PHP + JavaScript Static Data Demo

PROJECT GOAL

Build a working Accounting System demo in Laravel PHP with JavaScript static data. The complete flow from transaction entry to posting, ledger updates, and financial reports must behave interactively, but Version 1 must not require a real database. The system must be usable for live demonstrations and must not contain non-working placeholder buttons.

Project Type	OJT / Internship Development Project
Recommended Stack	Laravel PHP + Blade + JavaScript + Tailwind CSS or Bootstrap; static JS/JSON demo data only (no MySQL/database in Version 1)
Priority	Functional demo workflow first, static data architecture second, visual polish third
Target	Demo-ready MVP with static sample data and working client-side accounting calculations
Prepared For	OJT Developers / Interns

Version 1.1 | Static Demo / Mockup Specification

1. Project Overview

The OJT team will design and develop a functional web-based Accounting System intended for business demonstration and internal testing. Version 1 must be built with Laravel PHP and Blade for the application structure, routes, layouts, and reusable views, while all accounting records and sample transactions are supplied as static JavaScript objects or JSON-style data. No real database is required in this phase. The application should simulate a realistic accounting workflow while remaining simple enough for non-accountants to understand.

1.1 Main Objectives

- [] Create a complete accounting dashboard with useful business and financial summaries.
- [] Allow users to record, edit, review, approve, post, and reverse accounting transactions.
- [] Automatically update journals, ledgers, account balances, and reports after transactions are posted.
- [] Provide Accounts Receivable and Accounts Payable workflows.
- [] Generate core financial statements from recorded transactions.
- [] Simulate user roles, permissions, validation, and audit history for demo purposes using static configuration and JavaScript state.
- [] Use realistic static JavaScript/JSON sample data so every module can be demonstrated immediately without MySQL or another database.
- [] Ensure every visible action button either works or is clearly disabled with an explanation.

1.2 Project Rules

- Do not build static pages that only look functional. Forms, buttons, filters, modals, status changes, and calculations must work.
- Transactions that affect accounting balances must use one consistent JavaScript posting/calculation process so all screens derive from the same demo data.
- Posted demo transactions should not be silently edited during a session. Simulate reversal or controlled adjustment flows in JavaScript.
- Use client-side JavaScript validation to prevent incomplete, duplicated, or obviously invalid demo transactions.
- Use clean, professional business UI. Avoid excessive gradients, oversized cards, and decorative effects that reduce readability.
- Provide empty states, loading states, success messages, validation errors, and confirmation prompts where appropriate.

1.3 Version 1 Technical Architecture - No Database

Version 1 is intentionally a static-data demonstration. Do not configure MySQL, PostgreSQL, SQLite, or another production database. Do not create production migrations or rely on Eloquent persistence for the demo data.

[] Laravel PHP provides routes, Blade layouts/components, navigation, page rendering, and reusable server-side structure.

[] JavaScript modules provide the demo records, calculations, filters, status changes, posting logic, charts, and temporary transaction state.

[] Store initial demo datasets in organized JS files such as data/accounts.js, data/customers.js, data/invoices.js, data/bills.js, and data/journals.js, or equivalent JSON-style modules.

[] Changes may exist only until page refresh, or may optionally use sessionStorage/localStorage for convenience. If localStorage is used, include a visible Reset Demo Data action.

[] Do not send or store real customer, vendor, employee, bank, tax, or company-sensitive information in the demo.

[] All dashboards and reports must be computed from the demo arrays so the displayed totals change when users create, post, pay, reverse, or filter transactions.

[] Use mock authentication/role switching or fixed demo accounts without a database. Clearly label this as demonstration access control, not production security.

[] Design the code so the static data service can later be replaced by Laravel controllers/API endpoints and Eloquent models without redesigning the entire UI.

2. User Roles and Access

Role	Main Access	Restrictions
Administrator	Full system access, users, roles, settings, accounting configuration, approvals, reports (simulated for Version 1)	None except protected system rules
Accountant	Transactions, journals, ledgers, AR/AP, bank, reports, posting and adjustments (simulated for Version 1)	No user administration unless granted
Encoder / Staff	Create drafts, encode invoices, bills, receipts, payments and expenses (simulated for Version 1)	Cannot approve/post protected transactions
Viewer / Auditor	View dashboards, reports, posted entries and audit logs (simulated for Version 1)	Read-only; no create/edit/delete

ACCESS CONTROL REQUIREMENT

For Version 1, hide or disable actions based on the selected mock role using JavaScript/static role configuration. This is for demonstration only and is not production-grade authorization. Real Laravel authentication, policies, middleware, and database-backed users belong to Phase 2.

3. Required System Modules

#	Module	Minimum Requirement
1	Dashboard	KPIs, cash position, receivables, payables, revenue/expense summaries, recent activity

2	Chart of Accounts	Account codes, names, types, status, opening balance setup
3	Journal Entries	Balanced debit/credit entry, draft/review/post/reverse workflow
4	General Ledger	Account-level transaction history, running balance, filters
5	Accounts Receivable	Customers, invoices, receipts, balances, aging
6	Accounts Payable	Vendors, bills, payments, balances, aging
7	Cash and Bank	Cash accounts, bank accounts, deposits, withdrawals, transfers, reconciliation view
8	Sales and Revenue	Sales invoices and revenue transaction source documents
9	Expenses	Expense recording, categories/accounts, payment status, attachments
10	Financial Reports	Trial Balance, Income Statement, Balance Sheet, Cash Flow summary
11	Tax / VAT Settings	Configurable tax rates and tax summary for demo purposes
12	Audit Trail	Simulated in-session activity log: who/role, action, timestamp, record affected, before/after where practical
13	Users and Settings	Static demo users/roles, company information, fiscal year, numbering and preferences; no user database required

4. Dashboard Requirements

The dashboard should answer: How much cash is available? How much is collectible? How much is owed? Is the business profitable for the selected period?

4.1 Required KPI Cards

- Cash on Hand / Bank Balance
- Total Accounts Receivable
- Total Accounts Payable
- Revenue - current month
- Expenses - current month
- Net Income - current month
- Overdue Receivables
- Overdue Payables

4.2 Dashboard Visuals

- Monthly Revenue vs Expenses chart
- Cash movement chart or summary
- Receivables aging breakdown
- Payables aging breakdown
- Recent journal entries
- Recent customer payments
- Recent vendor payments

4.3 Dashboard Filters

- ☐ Date range
- ☐ This month / quarter / year shortcuts
- ☐ Company or branch filter if enabled later
- ☐ Refresh/recalculate action using the current JavaScript demo state
- ☐ View details link from each KPI to its source module

5. Chart of Accounts

5.1 Account Fields

Field	Example	Requirement
Account Code	1000	Required and unique
Account Name	Cash on Hand	Required
Account Type	Asset	Asset, Liability, Equity, Revenue, Expense
Sub-Type	Current Asset	Configurable
Parent Account	Cash and Cash Equivalents	Optional
Normal Balance	Debit	Derived or configured
Opening Balance	25,000.00	Optional setup value

Status	Active	Inactive accounts cannot be used for new entries
--------	--------	--

5.2 Required Actions

- ☐ Add account
- ☐ Edit account
- ☐ Deactivate account
- ☐ Search and filter by type/status
- ☐ View account ledger
- ☐ Prevent duplicate account codes
- ☐ Simulate deletion blocking if an account is referenced by posted demo transactions

6. Journal Entries

CORE ACCOUNTING RULE

Every journal entry must balance: Total Debits = Total Credits. The Post button must be disabled or blocked until the entry is balanced and valid.

6.1 Journal Entry Header

- Journal number (auto-generated)
- Transaction date
- Reference number
- Description / memo
- Source type (Manual, Invoice, Payment, Bill, Expense, etc.)
- Status (Draft, For Review, Posted, Reversed)
- Created by / reviewed by / posted by

6.2 Journal Lines

- Account
- Line description
- Debit amount
- Credit amount
- Customer or vendor reference when applicable
- Optional cost center / project tag for future expansion

6.3 Actions and Workflow

Status	Allowed Actions	Expected Behavior
Draft	Save, edit, delete, submit for review	No effect on official balances
For Review	Approve, return to draft	Reviewer checks supporting details
Posted	View, print, reverse	Updates ledger and financial reports
Reversed	View linked reversal	Original remains in history; reversal offsets balances

7. General Ledger and Trial Balance

7.1 General Ledger

- ☐ Select an account and date range
- ☐ Show beginning balance
- ☐ Show each posted transaction with date, reference, debit, credit, and running balance
- ☐ Link each ledger line to its source transaction
- ☐ Export or print ledger
- ☐ Do not include draft demo transactions in calculated posted balances

7.2 Trial Balance

- ☐ Display account code/name, debit balance, credit balance
- ☐ Filter by period
- ☐ Total debit must equal total credit
- ☐ Click an account to drill down to General Ledger
- ☐ Print/export-friendly layout

8. Accounts Receivable (AR)

8.1 Customer Management

- Customer code
- Business/customer name
- Contact person
- Email and phone
- Billing address
- Tax identification field (optional for demo)
- Credit terms
- Opening balance
- Status

8.2 Sales Invoice

Field / Action	Requirement
Invoice No.	Auto-generated, unique
Customer	Required
Invoice Date / Due Date	Required and used for aging
Line Items	Description, quantity, unit price, tax, amount
Discount	Optional
Subtotal / Tax / Total	Automatically calculated
Status	Draft, Issued/Posted, Partially Paid, Paid, Overdue, Voided
Print View	Professional invoice layout
Post	Creates accounting entry and customer balance

8.3 Customer Receipt / Payment

- ☐ Select customer
- ☐ Enter payment date and amount
- ☐ Select cash/bank account
- ☐ Apply payment to one or more invoices
- ☐ Allow partial payments
- ☐ Automatically update invoice remaining balance
- ☐ Generate receipt number
- ☐ Post corresponding journal entry

8.4 AR Aging

- Current
- 1-30 days
- 31-60 days
- 61-90 days

- Over 90 days
- Total outstanding per customer

9. Accounts Payable (AP)

9.1 Vendor Management

- Vendor code and name
- Contact details
- Address
- Payment terms
- Tax information field
- Opening balance
- Status

9.2 Vendor Bill

- ☐ Bill/reference number
- ☐ Vendor
- ☐ Bill date and due date
- ☐ Expense or asset account lines
- ☐ Tax calculation
- ☐ Attachments for invoice/receipt image or PDF
- ☐ Draft/posted/payment status
- ☐ Posting creates AP and corresponding debit entry

9.3 Vendor Payment

- ☐ Select vendor and open bills
- ☐ Allow full or partial payment
- ☐ Select cash/bank account
- ☐ Apply one payment to multiple bills
- ☐ Update remaining balance
- ☐ Generate payment reference
- ☐ Create journal entry automatically

9.4 AP Aging

- Current
- 1-30 days
- 31-60 days
- 61-90 days
- Over 90 days
- Total payable per vendor

10. Cash and Bank Management

- ☐ Maintain multiple cash and bank accounts
- ☐ Record cash deposits and withdrawals

- ☐ Transfer funds between accounts
- ☐ Record bank charges and interest
- ☐ Show running balance by account
- ☐ Provide a bank reconciliation screen with Book Balance, Bank Statement Balance, Uncleared Items, and Difference
- ☐ Allow marking transactions as cleared/reconciled
- ☐ Simulate reconciliation history during the current demo session

MVP NOTE

The reconciliation module may use manually entered statement balances and static bank transactions. Direct bank API integration, statement persistence, and real bank connectivity are outside Version 1.

11. Expense Management

- ☐ Create expense transaction with date, payee, category/account, amount, tax, payment method, and memo
- ☐ Demo file picker/preview for supporting receipt/document; no permanent server storage required in Version 1
- ☐ Mark as paid or unpaid
- ☐ Link paid expenses to cash/bank account
- ☐ Post accounting entry automatically
- ☐ Search by date, payee, category, status, and amount range
- ☐ Print or export expense list

12. Tax / VAT Demonstration Settings

The system should support configurable tax rates for demonstration. Store demo tax codes/rates in JavaScript configuration or a static JSON file. This project is not a substitute for tax compliance software, so tax rules must be clearly labeled as demo configuration.

- ☐ Create tax codes (for example: VAT, zero-rated, exempt, withholding placeholder)
- ☐ Assign default tax rate
- ☐ Apply tax to invoice or bill lines
- ☐ Show input/output tax summaries
- ☐ Filter tax summary by period
- ☐ Keep tax logic configurable instead of hard-coding a single business rule

13. Financial Reports

Report	Minimum Contents	Required Filters
Trial Balance	All account balances; debit and credit totals	As-of date / period
Income Statement	Revenue, cost/expense groups, net income	Date range
Balance Sheet	Assets, liabilities, equity; totals must balance	As-of date
Cash Flow Summary	Operating, investing, financing or simplified inflow/outflow groups	Date range
General Ledger	Detailed account activity and running balance	Account + date range
AR Aging	Outstanding customer balances by aging bucket	As-of date, customer
AP Aging	Outstanding vendor balances by aging bucket	As-of date, vendor
Sales Report	Invoices, collections, customer totals	Date, customer, status
Expense Report	Expense details and totals by category	Date, category, status
Tax Summary	Taxable amounts and configured tax totals	Date range, tax code

13.1 Report Behavior

- ☐ Use only demo transactions with Posted status when calculating financial statements
- ☐ Provide print-friendly format
- ☐ Allow client-side CSV export where practical; Excel/PDF export may be added later
- ☐ Show report generation date/time and selected filters
- ☐ Provide drill-down links to transaction details
- ☐ Display "No data" clearly when a period has no records

14. Automatic Accounting Entries

Source transactions should automatically generate simulated journal entries in JavaScript based on configured account mappings. The OJT team should use the following examples as the initial implementation. These entries only modify the active demo state and are not saved to a database.

Business Event	Debit	Credit
Cash sale	Cash / Bank	Sales Revenue (+ Output Tax if configured)
Credit sale / invoice	Accounts Receivable	Sales Revenue (+ Output Tax if configured)
Customer payment	Cash / Bank	Accounts Receivable

Expense paid immediately	Expense Account (+ Input Tax if configured)	Cash / Bank
Vendor bill on credit	Expense / Asset (+ Input Tax if configured)	Accounts Payable
Vendor payment	Accounts Payable	Cash / Bank
Bank transfer	Destination Bank	Source Bank
Owner capital contribution	Cash / Bank	Owner Equity / Capital

IMPORTANT

Do not duplicate simulated journal entries. A source document marked as posted in the current JavaScript demo state must store/link its journalEntryId and must not post again unless the demo reversal/void flow is used.

15. Suggested Static JavaScript Data Objects

JS Object / Module	Purpose
users / roles / permissions	Static demo accounts and role-access rules
accounts	Chart of Accounts array
journalEntries	Journal header/status/source array
journalLines	Debit/credit line array
customers	Customer master array
invoices / invoiceLines	Accounts Receivable demo records
customerPayments / paymentAllocations	Collections and invoice application
vendors	Vendor master array
bills / billLines	Accounts Payable demo records
vendorPayments / vendorPaymentAllocations	Vendor payment application
cashAccounts / bankAccounts	Cash and bank master arrays
bankTransactions	Deposits, withdrawals, transfers, charges
expenses	Expense demo records
taxCodes	Configurable demo tax setup
attachments	Client-side file metadata/preview only; no permanent upload required
auditLogs	In-memory demo activity history
settings	Static company/accounting configuration object

16. Business Rules and Validations

- [] Journal entries cannot post unless total debit equals total credit.
- [] Amounts must be non-negative unless the specific transaction type supports credit/adjustment logic.
- [] Posted demo records cannot be removed through normal actions during the active demo session; use void/reversal simulation.
- [] Duplicate invoice numbers, bill references, journal numbers, receipt numbers, and payment references should be detected where applicable.
- [] Invoice/bill due date cannot normally be earlier than the document date.
- [] Payment allocation cannot exceed the remaining open balance unless overpayment support is intentionally implemented.
- [] Inactive accounts, customers, and vendors cannot be selected for new transactions.
- [] Every posted source document in the JavaScript demo state must link to exactly one simulated journal entry.
- [] Reversal must create an offsetting transaction and preserve both records.
- [] Financial reports and dashboard KPIs must be calculated from the static/demo transaction arrays, not from separate hard-coded totals.

17. Required Pages and Navigation

Menu	Pages / Screens
Dashboard	Overview
Accounting	Chart of Accounts; Journal Entries; General Ledger; Trial Balance
Sales / AR	Customers; Invoices; Customer Payments; AR Aging
Purchases / AP	Vendors; Bills; Vendor Payments; AP Aging
Cash & Bank	Accounts; Transactions; Transfers; Reconciliation
Expenses	Expense List; New Expense; Expense Categories
Reports	Income Statement; Balance Sheet; Cash Flow; Sales; Expenses; Tax Summary
Administration	Users; Roles & Permissions; Audit Logs; Company Settings; Accounting Settings

17.1 Every List Page Must Include

- ☐ Search
- ☐ Relevant filters
- ☐ Sortable columns where useful
- ☐ Client-side pagination or a simple paginated presentation for larger demo lists
- ☐ View action
- ☐ Edit action when allowed
- ☐ Delete/void/deactivate action when allowed by the simulated workflow
- ☐ Create New button
- ☐ Status badges
- ☐ Empty state
- ☐ Export/print if applicable

17.2 Every Form Must Include

- ☐ Clear field labels
- ☐ Required-field indicators
- ☐ Inline validation messages
- ☐ Save Draft to the current JavaScript demo state where applicable
- ☐ Save/Submit action
- ☐ Cancel/Back action
- ☐ Confirmation for irreversible actions
- ☐ Success or failure feedback
- ☐ Protection against accidental double submission

18. Required Static Demo Data

The delivered system must be immediately demo-ready after setup. Provide realistic sample records in organized static JavaScript/JSON data files instead of leaving pages empty.

Static Data Type	Minimum Sample
Chart of Accounts	25-40 accounts covering assets, liabilities, equity, revenue, expenses
Customers	8-10
Vendors	6-8
Sales Invoices	15+ with mixed paid/partial/unpaid/overdue statuses
Vendor Bills	12+ with mixed statuses
Customer Payments	8+
Vendor Payments	6+
Expenses	15+
Bank/Cash Transactions	20+
Manual Journal Entries	5+
Users	At least one per role

19. Suggested OJT Work Plan

Phase	Focus	Deliverable
Phase 1	Laravel project setup, Blade layouts, routes, reusable components, static JS data service, mock roles	Running base demo with no database connection
Phase 2	Chart of Accounts, journals, JavaScript posting engine, ledger calculations	Core accounting demo engine
Phase 3	Customers, invoices, receipts, AR aging using static arrays	Working Accounts Receivable demo
Phase 4	Vendors, bills, payments, AP aging using static arrays	Working Accounts Payable demo
Phase 5	Cash/bank, transfers, expenses, tax settings, client-side document previews	Operational transaction demo modules
Phase 6	Financial reports, dashboard KPIs, charts calculated from demo state	End-to-end reporting demo

Phase 7	Mock audit logs, validation, role simulation, reset demo data, sample scenarios	Demo-ready controls and repeatability
Phase 8	Testing, bug fixing, documentation, final live demo	Final OJT submission

20. Suggested Team Assignment

Role	Responsibilities
OJT Lead / Coordinator	Task assignment, code review, integration, progress tracking
Backend Developer	Laravel routes/controllers where needed, Blade structure, static data service conventions, future API/database-ready architecture; no production database in Version 1
Frontend Developer	Blade pages, JavaScript interactions, forms, tables, filters, modals, charts, responsive UI, in-memory/static data updates
Accounting Logic / QA	Test JavaScript debit-credit logic, workflow statuses, derived balances, reports, and edge cases
Documentation / QA	Test cases, user guide, screenshots, demo script, bug tracking

21. Minimum Acceptance Test

THE DEMO MUST PASS THIS SCENARIO

If the following end-to-end scenario works correctly, the system demonstrates that its main accounting workflow is connected.

1. Create a new customer.
2. Create and post a credit sales invoice for that customer.
3. Confirm Accounts Receivable and Revenue increased through the generated journal entry.
4. Confirm the customer invoice appears in AR Aging.
5. Record a partial customer payment and apply it to the invoice.
6. Confirm Cash/Bank increases and Accounts Receivable decreases.
7. Create a vendor and post a vendor bill.
8. Confirm Accounts Payable and the selected Expense/Asset account increase.
9. Pay the vendor bill and confirm Accounts Payable decreases and Cash/Bank decreases.
10. Open Trial Balance and confirm total debits equal total credits.
11. Open Income Statement and Balance Sheet and verify the transactions are reflected.
12. Reverse one posted journal and confirm the reversal appears in the audit history and reports update correctly.

22. Quality Assurance Checklist

- ☐ No broken links or pages in the sidebar/navigation.
- ☐ No button is present purely for decoration.
- ☐ No duplicate transaction occurs when a submit/post button is clicked multiple times.
- ☐ All totals recalculate correctly after create, update, payment, post, reverse, or void actions.
- ☐ Role restrictions are tested using different user accounts.
- ☐ Forms preserve user-entered values when validation fails.
- ☐ Desktop and mobile/tablet layouts remain usable.
- ☐ Currency values use consistent decimal formatting.
- ☐ Date/time display is consistent throughout the system.
- ☐ Tables remain readable with long names and many rows.
- ☐ Financial reports match the General Ledger balances.
- ☐ Reset Demo Data restores the original static JavaScript/JSON dataset to a clean demo state.

23. Final OJT Deliverables

- ☐ Complete source code in the designated repository.
- ☐ Organized static JavaScript/JSON data modules plus a documented demo data service/state layer; no database migrations are required for Version 1.
- ☐ README with setup and run instructions.
- ☐ Demo accounts and passwords documented securely for the supervisor.
- ☐ Short user guide covering the main workflows.
- ☐ Testing checklist with passed/failed status and resolved bugs.
- ☐ Screenshots or short demo recording of the completed system.
- ☐ Final presentation/demo showing the acceptance scenario in Section 21.
- ☐ Known limitations and recommended Phase 2 improvements.

24. Production Upgrade / Optional Phase 2 Features

- ☐ Connect MySQL/PostgreSQL and create proper Laravel migrations.
 - ☐ Replace static JavaScript records with Eloquent models and database repositories/services.
 - ☐ Add Laravel authentication, policies, middleware, server-side validation, and database-backed audit logs.
 - ☐ Add persistent file storage for receipts and attachments.
 - ☐ Add server-side transactions/locking and production-grade accounting posting controls.
- Inventory and Cost of Goods Sold
 - Purchase Orders and Sales Orders
 - Budgeting and budget-vs-actual reports
 - Cost centers / departments / projects
 - Recurring transactions
 - Multi-branch or multi-company support

- Fixed asset register and depreciation
- Payroll accounting integration
- Bank statement CSV import
- Approval rules based on amount
- Advanced tax reports
- API integrations and webhooks
- PDF/Excel report export enhancements
- Backup/restore tools
- Notification center and email reminders

PRIORITY REMINDER

Finish the core accounting workflow first. A smaller system with correct posting, balances, and reports is better than a large system full of unfinished modules.

25. Supervisor Review and Sign-Off

Review Item	Status / Notes
Core modules completed	
Acceptance test passed	
Permissions tested	
Reports validated	
Demo data complete	
Documentation complete	
Final demo approved	
No real database used in Version 1	

OJT Lead / Developer	Supervisor / Reviewer
Signature / Date: _____	Signature / Date: _____

END OF PROJECT BRIEF